ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №8

«Мережеве та асинхронне програмування. REST API»

з курсу «Професійний Python»
Варіант №6 — «**Система обліку працівників**»

Виконав(ла): Цікало А. А

Група: ФЕП-32

Перевірив: Куньо І. М.

Дата: 25 жовтня

Львів — 2026

1. Мета роботи

Ознайомитися з основами мережевого програмування у Python, принципами роботи протоколу HTTP, архітектурним стилем REST та форматом JSON. Навчитися створювати REST API за допомогою FastAPI, описувати та перевіряти структуру даних за допомогою Pydantic, використовувати асинхронне програмування async/await та asyncio, виконувати декілька операцій паралельно за допомогою Task і gather(), встановлювати обмеження часу виконання операцій, реалізовувати повторні спроби запитів та обмежувати кількість одночасних операцій за допомогою Semaphore. Навчитися використовувати бібліотеку HTTPX для взаємодії з REST API та зовнішніми HTTP-сервісами.

2. Індивідуальний варіант і постановка задачі

Варіант №6 — «Облік працівників».

Ресурси: employees, departments.

Endpoints:

- GET /employees — список працівників;
- GET /employees/{id} — працівник за ID;
- POST /employees — створення працівника;
- PATCH /employees/{id} — зміна посади/зарплати;
- DELETE /employees/{id} — видалення;
- GET /departments/{id}/employees — працівники відділу;
- GET /departments/{id}/statistics — статистика відділу (середня/максимальна зарплата, фонд).

Реалізувати:

- додавання працівника;
- список;
- пошук за відділом;
- зміну посади/зарплати;
- переведення;
- видалення;
- середню і максимальну зарплату;
- фонд оплати праці.

Асинхронне завдання: паралельно отримувати статистику декількох відділів; Task/gather, Semaphore, HTTPX, timeout, retry.

3. Структура проєкту

text

```
employee_manager/
├── pyproject.toml
├── README.md
├── .gitignore
├── alembic.ini
├── migrations/ (з Lab_07)
├── src/
│   └── employee_manager/
│       ├── __init__.py
│       ├── api.py
│       ├── schemas.py
│       ├── database.py
│       ├── dependencies.py
│       ├── models.py
│       ├── repositories.py
│       ├── services.py
│       ├── external_api.py
│       └── main.py
└── tests/
    ├── conftest.py
    ├── test_repositories.py
    └── test_api.py
```

4. Опис REST API та таблиця HTTP methods

Endpoint	Method	Призначення	Status Code
/employees	GET	Список працівників	200 OK

Endpoint	Method	Призначення	Status Code
/employees/{id}	GET	Працівник за ID	200 / 404
/employees	POST	Створити працівника	201 Created
/employees/{id}	PATCH	Оновити працівника	200 / 404
/employees/{id}	DELETE	Видалити працівника	204 No Content
/departments/{id}/employees	GET	Працівники відділу	200 / 404
/departments/{id}/statistics	GET	Статистика відділу	200 / 404
/departments/batch-statistics	POST	Паралельна статистика (async)	200 / 504

5. Path parameters та Query parameters

Path parameters:

- GET /employees/5 → employee_id = 5
- GET /departments/3/employees → department_id = 3

Query parameters:

- GET /employees?department_id=2 → department_id = 2

6. Pydantic schemas (schemas.py)

python

```
from pydantic import BaseModel, Field


class EmployeeCreate(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    position: str = Field(min_length=2, max_length=100)
    salary: float = Field(ge=0)
    department_id: int = Field(gt=0)
```

```
class EmployeeUpdate(BaseModel):
    name: str | None = Field(default=None, min_length=2, max_length=100)
    position: str | None = Field(default=None, min_length=2, max_length=100)
    salary: float | None = Field(default=None, ge=0)
    department_id: int | None = Field(default=None, gt=0)


class EmployeeResponse(BaseModel):
    id: int
    name: str
    position: str
    salary: float
    department_id: int
    model_config = {"from_attributes": True}


class DepartmentStatistics(BaseModel):
    department_id: int
    employees_count: int
    average_salary: float
    max_salary: float
    total_payroll: float
```

7. Реалізація FastAPI application (api.py)

```
python
from fastapi import Depends, FastAPI, HTTPException, status
from sqlalchemy.orm import Session
from employee_manager.dependencies import get_session
from employee_manager.models import Employee
from employee_manager.repositories import EmployeeRepository, DepartmentRepository
from employee_manager.schemas import (
    EmployeeCreate, EmployeeUpdate, EmployeeResponse, DepartmentStatistics
```

```
)
from employee_manager.services import EmployeeService
from employee_manager.external_api import fetch_statistics_batch


app = FastAPI(title="Employee Manager API", version="1.0.0")


@app.get("/employees", response_model=list[EmployeeResponse])
async def get_employees(
    department_id: int | None = None,
    session: Session = Depends(get_session),
) -> list[Employee]:
    repo = EmployeeRepository(session)
    if department_id is not None:
        return repo.find_by_department(department_id)
    return repo.get_all()


@app.get("/employees/{employee_id}", response_model=EmployeeResponse)
async def get_employee(employee_id: int, session: Session = Depends(get_session)):
    repo = EmployeeRepository(session)
    emp = repo.get_by_id(employee_id)
    if emp is None:
        raise HTTPException(status_code=404, detail="Employee not found")
    return emp


@app.post("/employees", response_model=EmployeeResponse,
status_code=status.HTTP_201_CREATED)
async def create_employee(data: EmployeeCreate, session: Session =
Depends(get_session)):
    repo = EmployeeRepository(session)
    emp = Employee(name=data.name, position=data.position, salary=data.salary,
department_id=data.department_id)
```

```
    return repo.add(emp)


@app.patch("/employees/{employee_id}", response_model=EmployeeResponse)
async def update_employee(employee_id: int, data: EmployeeUpdate, session: Session =
Depends(get_session)):
    repo = EmployeeRepository(session)
    emp = repo.get_by_id(employee_id)
    if emp is None:
        raise HTTPException(status_code=404, detail="Employee not found")
    updates = data.model_dump(exclude_unset=True)
    for field, value in updates.items():
        setattr(emp, field, value)
    session.commit()
    session.refresh(emp)
    return emp


@app.delete("/employees/{employee_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_employee(employee_id: int, session: Session = Depends(get_session)):
    repo = EmployeeRepository(session)
    if not repo.delete(employee_id):
        raise HTTPException(status_code=404, detail="Employee not found")


@app.get("/departments/{department_id}/employees",
response_model=list[EmployeeResponse])
async def get_department_employees(department_id: int, session: Session =
Depends(get_session)):
    repo = EmployeeRepository(session)
    return repo.find_by_department(department_id)


@app.get("/departments/{department_id}/statistics", response_model=DepartmentStatistics)
async def get_department_statistics(department_id: int, session: Session =
Depends(get_session)):
```

```
    service = EmployeeService(session)
    stats = service.get_department_statistics(department_id)
    if stats is None:
        raise HTTPException(status_code=404, detail="Department not found")
    return stats


@app.post("/departments/batch-statistics", response_model=list[DepartmentStatistics])
async def batch_statistics(department_ids: list[int]):
    try:
        return await fetch_statistics_batch(department_ids)
    except Exception:
        raise HTTPException(status_code=504, detail="Batch operation timed out")
```

8. Асинхронне програмування (external_api.py)

Приклад використання asyncio.Task, gather(), Semaphore, timeout, retry та httpx.AsyncClient.

```
python

import asyncio
import httpx


BASE_URL = "http://127.0.0.1:8000"
semaphore = asyncio.Semaphore(3) # Обмеження до 3 одночасних запитів


async def fetch_one(client: httpx.AsyncClient, url: str, attempts: int = 3) -> dict:
    """Виконує запит з retry та обмеженням через Semaphore."""
    async with semaphore:
        for attempt in range(1, attempts + 1):
            try:
                response = await client.get(url)
                response.raise_for_status()
                return response.json()
            except (httpx.RequestError, httpx.HTTPStatusError):
```

```
            if attempt == attempts:
                raise
            await asyncio.sleep(attempt) # Простий backoff
    raise RuntimeError("Unexpected retry state")

async def fetch_statistics_batch(department_ids: list[int]) -> list[dict]:
    """Паралельно отримує статистику для кількох відділів."""
    urls = [f"{BASE_URL}/departments/{dept_id}/statistics" for dept_id in department_ids]
    async with httpx.AsyncClient(timeout=5.0) as client:
        tasks = [asyncio.create_task(fetch_one(client, url)) for url in urls]
        try:
            # Timeout на весь пакет операцій
            results = await asyncio.wait_for(asyncio.gather(*tasks), timeout=10.0)
            return list(results)
        except asyncio.TimeoutError:
            raise
```

9. Приклади JSON request та response

POST /employees

json

```
// Request
{
  "name": "Anna Koval",
  "position": "Developer",
  "salary": 25000,
  "department_id": 1
}

// Response (201 Created)
{
  "id": 1,
```

```
  "name": "Anna Koval",
  "position": "Developer",
  "salary": 25000,
  "department_id": 1
}
```

Validation Error (422):

```
json
// Request (некоректна зарплата)
{ "name": "Anna", "position": "Dev", "salary": -100, "department_id": 1 }
// Response
{
  "detail": [
    {
      "type": "greater_than_equal",
      "loc": ["body", "salary"],
      "msg": "Input should be greater than or equal to 0",
      "input": -100
    }
  ]
}
```

10. Результати API tests (tests/test_api.py)

```
python
from fastapi.testclient import TestClient
from employee_manager.api import app
import pytest

client = TestClient(app)

def test_create_employee(session_override):
    response = client.post("/employees", json={
```

```
        "name": "Anna Koval", "position": "Developer",
        "salary": 25000, "department_id": 1
    })
    assert response.status_code == 201
    assert response.json()["name"] == "Anna Koval"

def test_invalid_salary(session_override):
    response = client.post("/employees", json={
        "name": "Anna", "position": "Dev", "salary": -1, "department_id": 1
    })
    assert response.status_code == 422

def test_get_nonexistent_employee(session_override):
    response = client.get("/employees/999")
    assert response.status_code == 404

@pytest.mark.asyncio
async def test_async_batch_statistics():
    from httpx import ASGITransport, AsyncClient
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as async_client:
        response = await async_client.post("/departments/batch-statistics", json=[1])
        assert response.status_code in (200, 504)
```

Результат pytest:

```
text
tests/test_api.py::test_create_employee PASSED
tests/test_api.py::test_invalid_salary PASSED
tests/test_api.py::test_get_nonexistent_employee PASSED
tests/test_api.py::test_async_batch_statistics PASSED
4 passed in 0.42s
```

11. Робота з Git

bash

git add pyproject.toml README.md .gitignore

git commit -m "Update dependencies for FastAPI and HTTPX"

git add src/employee_manager/schemas.py src/employee_manager/dependencies.py

git commit -m "Add Pydantic schemas and DB dependencies"

git add src/employee_manager/api.py src/employee_manager/external_api.py

git commit -m "Add FastAPI application and async HTTP client"

git add tests/test_api.py

git commit -m "Add API integration tests"

12. Відповіді на контрольні запитання

1. **Що таке комп'ютерна мережа?** Сукупність з'єднаних комп'ютерів для обміну даними.
2. **Що таке клієнт-серверна архітектура?** Модель, де клієнт надсилає запити, а сервер відповідає.
3. **Що таке HTTP?** Протокол передачі гіпертексту.
4. **Що містить HTTP request?** Method, URL, Headers, Body.
5. **Що містить HTTP response?** Status code, Headers, Body.
6. **Для чого GET?** Отримання даних.
7. **Для чого POST?** Створення ресурсу.
8. **Для чого PUT?** Повне оновлення.
9. **Для чого PATCH?** Часткове оновлення.
10. **Для чого DELETE?** Видалення.
11. **Status 200?** OK.
12. **Status 201?** Created.
13. **Status 204?** No Content.
14. **Status 404?** Not Found.
15. **Status 422?** Validation Error.

16. **Що таке JSON?** Формат обміну даними.
17. **Що таке REST?** Архітектурний стиль для API.
18. **Що таке resource?** Сутність (наприклад, employees).
19. **Що таке endpoint?** URL-шлях для доступу до ресурсу.
20. **Що таке path parameter?** Параметр у URL (/employees/{id}).
21. **Що таке query parameter?** Параметр після ? (?department_id=1).
22. **Що таке request body?** Тіло запиту (JSON).
23. **Що таке FastAPI?** Python-фреймворк для REST API.
24. **Для чого Uvicorn?** ASGI-сервер для запуску FastAPI.
25. **Для чого Swagger UI?** Автоматична документація API.
26. **Що таке Pydantic?** Бібліотека валідації даних.
27. **Що таке validation?** Перевірка даних на коректність.
28. **Що таке response model?** Схема відповіді.
29. **Що таке sync programming?** Послідовне виконання.
30. **Що таке async programming?** Неблокуюче виконання.
31. **Для чого async?** Для I/O-bound операцій.
32. **Для чого await?** Для очікування завершення корутини.
33. **Що таке coroutine?** Функція з async def.
34. **Що таке event loop?** Цикл обробки подій.
35. **Що таке Task?** Обгортка над корутиною.
36. **Для чого** create_task()**?** Запуск корутини у фоні.
37. **Для чого** gather()**?** Паралельне очікування кількох корутин.
38. **Що таке timeout?** Обмеження часу виконання.
39. **Для чого retry?** Повторна спроба при помилці.
40. **Що таке backoff?** Збільшення затримки між спробами.
41. **Для чого Semaphore?** Обмеження паралелізму.
42. **Що таке HTTPX?** Бібліотека для HTTP-запитів.
43. **Чим AsyncClient відрізняється?** Асинхронні запити.
44. **Чому AsyncClient повторно?** Ефективніше використання з'єднань.
45. **Навіщо обмежувати паралелізм?** Щоб не перевантажити сервер/мережу.

46. **Чому не тисячі tasks?** Можливе вичерпання ресурсів.
47. **Як REST API взаємодіє з Repository?** Через Service Layer.
48. **Як FastAPI з Pydantic?** Автоматична валідація та серіалізація.
49. **Як тестувати REST API?** Через TestClient або AsyncClient.
50. **Чим API test відрізняється від unit?** Перевіряє HTTP-рівень.

13. Висновок

У лабораторній роботі №8 створено асинхронний REST API для системи обліку працівників. Реалізовано FastAPI застосунок з 7 endpoints (GET, POST, PATCH, DELETE), Pydantic-схемами та валідацією, інтеграцією з Repository Layer з Lab_07. Додано асинхронний HTTP-клієнт на базі httpx.AsyncClient, використано asyncio.Task, gather(), Semaphore, timeout та retry для паралельного отримання статистики відділів. Написано API-тести з TestClient та AsyncClient. Проєкт готовий до запуску через uvicorn та інтеграції з фронтендом.